

Software Engineering Professional Practice

ACRONYMS

ACM	Association for Computing Machinery
CCPA	The California Consumer Privacy Act
EEA	European Economic Area
ENAE	European Network for Accreditation of Engineering Education
EU	European Union
GDPR	General Data Protection Regulation
IEA	International Engineering Alliance
IEC	International Electrotechnical Commission
IEEE CS	IEEE Computer Society
IFIP	International Federation for Information Processing
IP	Intellectual Property
ISO	International Organization for Standardization
NDA	Nondisclosure Agreement
UI/UX	User Interface/User Experience
WIPO	World Intellectual Property Organization
WTO	World Trade Organization

INTRODUCTION

The Software Engineering Professional Practice knowledge area (KA) is concerned with the knowledge, skills, and attitudes software engineers must possess to practice software engineering in a professional, responsible and ethical manner. Because of the widespread

applications of software products in social and personal life, software product quality can profoundly affect personal well-being and societal harmony. Software engineers must handle unique engineering problems to produce software with known characteristics and reliability. This requirement calls for software engineers who possess the proper knowledge, skills, training, and experience in professional practice.

Professional practice refers to a way of conducting services to achieve certain standards or criteria in both the process of performing a service and the end product resulting from the service. These standards and criteria can include both technical and non-technical aspects. The concept of professional practice is especially applicable to professions with a generally accepted body of knowledge; code of ethics and professional conduct with penalties for violations; accepted processes for accreditation, certification, qualification, and licensing; and professional societies to provide and administer all these. Admission to these professional societies is often predicated on a prescribed combination of education and experience.

A software engineer maintains professional practice by performing all work following generally accepted practices, standards, and guidelines set forth by the applicable professional society, such as the Association for Computing Machinery (ACM), Institute for Electrical and Electronics Engineers (IEEE), International Federation for Information Processing (IFIP), IEEE Computer Society (IEEE CS), International Organization for Standardization/International Electrotechnical Commission (ISO/IEC), and ISO/IEC/IEEE provide internationally accepted software engineering standards. All of these

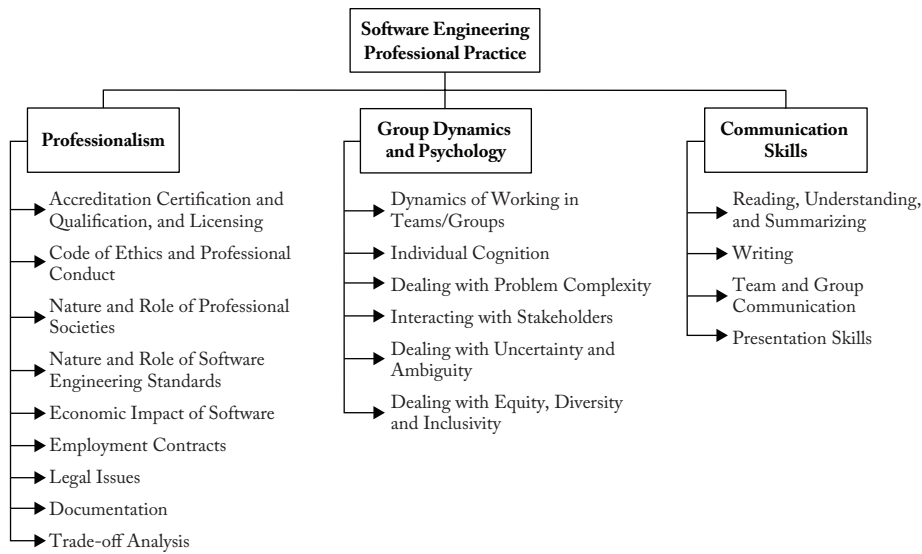


Figure 14.1. Breakdown of Topics for the Software Engineering Professional Practice KA

standards and guidelines comprise the foundation of the professional practice of software engineering.

BREAKDOWN OF TOPICS FOR SOFTWARE ENGINEERING PROFESSIONAL PRACTICE

The Software Engineering Professional Practice KA’s breakdown of topics is shown in Figure 14.1.

The subareas presented in this KA are professionalism, group dynamics and psychology, and communication skills.

1. Professionalism

A software engineer displays professionalism notably by adhering to a code of ethics and professional conduct and to standards and practices established by the engineer’s professional community.

One or more professional societies often represent a professional community, and this is the case for the engineering community. Those societies publish codes of ethics and professional conduct as well as criteria for

admittance to the community. Those criteria form the basis for accreditation and licensing activities and may determine engineering competence or negligence.

As software is used more widely and deeply in society, stakeholders’ requirements have likewise become wider and deeper. And as software has become socially vital, software engineers have worked to base the user interface/user experience (UI/UX) on socially inclusive concepts.

1.1. *Accreditation, Certification and Qualification, and Licensing*
[1*, cls4-s5, cls10] [2] [4*, c12s10] [6] [7] [8] [9]

1.1.1. *Accreditation*

Accreditation certifies an organization’s competency, authority, or credibility. Accredited schools or programs have shown that they adhere to particular standards and maintain certain qualities. In many countries, the basic means by which engineers acquire knowledge is by completing an accredited course of study. Often, the accreditation process is independent of the government and is performed by

private membership associations. There are two major global accreditation organizations. One is the International Engineering Alliance (IEA), of which the Washington Accord is a constituent. The other is the European Network for Accreditation of Engineering Education (ENAE), which administers EUR-ACE®, the label awarded to engineering degree programs at the bachelor's and master's levels, listed by the European Commission.

Although the accreditation process might differ for each country and jurisdiction, the general meaning is the same. Accreditation of an institution's course of study means "the accreditation body recognizes an educational institution as maintaining standards that qualify the graduates for admission to higher or more specialized institutions or professional practice."

1.1.2. *Certification and Qualification*

ISO/IEC 24773-1 Software and Systems Engineering — Certification of Software and Systems Engineering Professionals — Part 1: General Requirements [8] defines certification and qualification. ISO/IEC 24773-4 Software and Systems Engineering — Certification of Software and Systems Engineering Professionals — Part 4: Software engineering [9] elaborates requirements and recommendations for certification schemes based on ISO/IEC 24773-1, which are specific to the domain of software engineering. Certification contains recertification. Qualification is similar to certification but does not require re-qualification. Certification refers to the confirmation of a person's particular characteristics. A common type of certification is professional certification, which certifies a person as being able to complete an activity in a certain discipline at a stated level of competency. Professional certification can verify the holder's ability to meet professional standards and to apply professional judgment in solving or addressing problems. Professional certification can also verify prescribed knowledge, mastery of best practices and proven methodologies, and professional experience.

An engineer usually obtains certification by passing an examination in addition to meeting other experience-based criteria. Nongovernmental organizations, such as professional societies, often administer these examinations. In software engineering, certification testifies to one's capability as a software engineer.

The qualification and certification programs are designed to confirm a software engineer's knowledge of standard software engineering practices and to advance the engineer's career. A lack of qualification or certification does not exclude the individual from working as a software engineer. Qualification or certification in software engineering is voluntary. Most software engineers are not qualified or certified under any program.

1.1.3. *Licensing*

Licensing authorizes a person to perform certain activities and take responsibility for resultant engineering products. The noun *license* refers to both that authorization and the document recording that authorization. Governmental authorities or statutory bodies usually issue licenses.

Obtaining a license to practice requires an individual to meet a certain standard at a certain ability to practice or operate. Sometimes an entry-level requirement sets the minimum skills and capabilities to practice, and as the professional moves through their career, the required skills and capabilities change and evolve.

Engineers are licensed to protect the public from unqualified individuals. In some countries, no one can practice as a professional engineer unless licensed; further, no company may offer "engineering services" unless at least one licensed engineer is employed there.

1.2. *Codes of Ethics and Professional Conduct*

[1*, cls7-s9, c10s2, Appendix] [3*, c8]
[4*, cls2] [5*, c33][10] [11] [13*]

Codes of ethics and professional conduct describe the values and behavior that an engineer's professional practice and decisions

should embody. The professional community establishes a code of ethics and professional conduct. This code exists in the context of societal norms and local laws and is adjusted to agree with those norms and laws as needed. A code of ethics and professional conduct can offer guidance in the face of conflicting imperatives. More than one such code serves the professional engineering community. For example, in 1999, IEEE CS and ACM launched a joint Software Engineering Ethics and Professional Practices Task Force to publish a code of ethics. In 2018, ACM published its ACM Code of Ethics and Professional Conduct, and in 2020, IEEE published a revision of its Code of Ethics which was originally approved in 1912. Then, in 2021, IFIP published its Code of Ethics and Professional Conduct, adapted from ACM's Code of Ethics and Professional Conduct.

Once established, codes of ethics and professional conduct are enforced by the profession, as represented by professional societies or by a statutory body. Violations may be acts of commission, such as concealing inadequate work, disclosing confidential information, falsifying information, or misrepresenting abilities. They may also occur through omission, including failure to disclose risks or provide important information, failure to give proper credit or acknowledge references, and failure to represent client interests. Violations of a code of ethics and professional conduct may result in penalties and possible expulsion from professional status.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing, and maintenance of software a beneficial and respected profession. Following their commitment to the health, safety, and welfare of the public, software engineers shall adhere to the ten principles according to IEEE Code of Ethics adopted by the IEEE Board of Directions, June 2020.

Since the code of ethics and professional conduct may be introduced, modified, or replaced at any time, individual software

engineers are responsible for continuing their studies to stay current in their professional practice.

1.3. *Nature and Role of Professional Societies*

[1*, c2s3] [4*, c1s2] [5*, c35s1]

Professional societies comprise a mix of practitioners and academics. These societies define, advance, and regulate their corresponding professions. Professional societies help establish professional standards as well as codes of ethics and professional conduct. They also engage in related activities, which include the following:

- Establishing and promulgating a body of generally accepted knowledge
- Providing the basis for licensing, certifying, and accrediting
- Dispensing disciplinary actions
- Advancing the profession through conferences, training, publications, and standards

Participation in professional societies assists individual engineers in maintaining and sharpening their professional knowledge and relevancy and in expanding and maintaining their professional network.

1.4. *Nature and Role of Software Engineering Standards*

[1*, c10s2] [2] [4*] [5*, c32s6]

Software engineering standards cover a remarkable variety of topics. They provide guidelines for the practice of software engineering and for processes to be used during the development, maintenance, and support of software. By establishing a common body of knowledge and experience, software engineering standards establish a basis on which further guidelines may be developed. Appendix B of this *Guide* presents IEEE, ISO/IEC, and ISO/IEC/IEEE software engineering standards that support this *Guide's* KAs.

Standards are valuable sources of information about requirements and other

guidance that can support software engineers in everyday activities. Adherence to standards promotes discipline by enumerating minimal characteristics of products and practices. That discipline helps mitigate subconscious assumptions or overconfidence in a design. For these reasons, organizations performing software engineering activities often include conformance to standards as part of their organizational policies.

1.5. *Economic Impact of Software*

[3*, c1s1, c10s8] [4*, c1s1] [13*]

The software has economic effects at the individual, business, and societal levels. For example, software “success” may be determined by a product’s suitability for a recognized problem and by its effectiveness when applied to that problem. At the individual level, an engineer’s continuing employment may depend on their ability and willingness to interpret and execute tasks in meeting customers’ or employers’ needs and expectations. The customer’s or the employer’s financial situation may be positively or negatively affected by software purchases.

At the business level, software properly applied to a problem can eliminate months of work and translate to elevated profits or more effective organizations. Organizations that acquire or provide successful software may become a boon to the society in which they operate by providing both employment and improved services. However, the software’s development or acquisition costs can be considerable, like those of any major acquisition.

At the societal level, direct impacts of software success or failure include the avoidance or experience of accidents, interruptions, and loss of service. Indirect impacts include the success or failure of the organization that acquired or produced the software, increased or decreased societal productivity, harmonious or disruptive social order, and even the saving or loss of property or life. In addition, as digitalization progresses, easier and faster access to the information needed may bring higher social value.

1.6. *Employment Contracts*

[1*, c6, c7] [10] [11] [12]

Software engineering services may be provided under a variety of client-engineer relationships. For example, the work may be done through a company-to-customer supplier arrangement, an engineer-to-customer consultancy arrangement, a direct-hire, or even through volunteering. In these situations, the customer and supplier agree that a product or service will be provided in return for some consideration. Here, we are most concerned with engineer-to-customer arrangements and their attendant agreements or contracts, whether they are of the direct-hire or consultant variety, and the issues they typically address.

A common concern in software engineering contracts is confidentiality. Employers derive commercial advantage from intellectual property (IP), so they strive to protect that property from disclosure. Therefore, software engineers are often required to sign nondisclosure agreements (NDA) or IP agreements as a precondition to working. These agreements typically apply to information the software engineer could gain only through association with the customer. The terms of these agreements may extend past the association’s termination.

Another concern is IP ownership. Rights to software engineering assets — products, innovations, inventions, discoveries, and ideas — may reside with the employer or customer, under explicit contract terms or relevant laws, if those assets are obtained during the software engineer’s relationship with that employer or customer. Contracts differ in the ownership of assets created using non-employer-owned equipment or information.

Finally, contracts can also specify, among other elements:

- The location at which work is performed
- Standards to which that work will be held
- The system configuration used for development
- Limitations of the software engineer’s and employer’s liability

- A communication matrix and/or escalation plan
- Administrative details such as rates, frequency of compensation, working hours, and working conditions

1.7. *Legal Issues* [1*, c6, c11] [2] [3*, c5s3–c5s4] [4*, c12s3, c13s2]

Legal issues surrounding software engineering professional practice include matters related to standards, trademarks, patents, copyrights, trade secrets, professional liability, legal requirements, trade compliance, cybercrime, and data privacy. It is therefore beneficial to know these issues and their applicability. In addition, legal issues are jurisdictionally based, so software engineers must consult attorneys who specialize in the type and jurisdiction of any identified legal issues.

1.7.1. *Standards*

Adherence to standards provides a defense from legal action or allegations of malpractice.

1.7.2. *Trademarks*

A trademark relates to any word, name, symbol, or device used in business transactions. It is used “to indicate the source or origin of the goods.” Trademark protection protects names, logos, images, and packaging. However, if a name, image, or other trademarked asset becomes a generic term, trademark protection is nullified.

The World Intellectual Property Organization (WIPO) is the authority that frames the rules and regulations on trademarks. WIPO is the United Nations agency dedicated to protecting the use of IP as a means of stimulating innovation and creativity.

1.7.3. *Patents*

Patents protect an inventor’s right to manufacture and sell an idea. A patent consists of

exclusive rights granted by a sovereign government to an individual, group of individuals, or organization for a limited period. Patents are an old form of idea-ownership protection and date to the 15th century.

Application for a patent entails keeping and producing careful records of the process that led to the invention. In addition, patent attorneys help write patent disclosure claims in a manner most likely to protect the software engineer’s rights. Note that if inventions are made during a software engineering contract, ownership may belong to the employer or customer or be jointly held rather than belong to the software engineer.

Rules vary concerning what is and what is not patentable. In many countries, software code is not patentable, but software algorithms may be. Existing and filed patent applications can be found at WIPO.

1.7.4. *Copyrights*

Most legislations give exclusive rights of an original work to its creator, usually for a limited time, enacted as copyright. Copyrights protect the way an idea is presented — not the idea itself. For example, they may protect the particular wording of an account of a historical event, whereas the event itself is not protected. Copyrights are long-term and renewable. As a form of IP, they date to the 17th century.

1.7.5. *Trade Secrets*

In many countries, an intellectual asset such as a formula, algorithm, process, design, method, pattern, instrument, or compilation of information may be considered a *trade secret*, provided the asset is not generally known and may provide a business with some economic advantage. The “trade secret” designation provides legal protection if the asset is stolen. This protection is not subject to a time limit. However, if another party derives or discovers the same asset legally, then the asset is no longer protected and the other party will also possess all rights to use it.

1.7.6. Professional Liability

It is common for software engineers to be concerned with professional liability matters. As engineers provide services to a client or employer, it is crucial that they adhere to standards and generally accepted practices to protect themselves against allegations of or proceedings related to malpractice, negligence, or incompetence.

For engineers (including software engineers), professional liability is related to product liability. Under the laws and rules of their jurisdiction, engineers may be held accountable for failing to fully and conscientiously follow recommended practice; this is known as *negligence*. They may also be subject to laws governing *strict liability* and implied or express warranty, where, by selling the product, the engineer is held to warrant that the product is both suitable and safe for use. In some countries (e.g., in the US), *privity* (a doctrine under which one can sue only the person selling the product) is no longer a defense against liability actions.

Legal suits for liability can be brought under tort law in the US, allowing anyone who is harmed to recover their loss even if no guarantees were made. Because it is difficult to measure the suitability or safety of software, failure to take due care can be used to prove negligence on the part of software engineers. Engineers can defend themselves against such an allegation by showing that they followed standards and generally accepted practices in developing the product to be ready to consult with attorneys regarding the standard of care in any relevant jurisdiction to manage risks associated with product liability or professional liability.

1.7.7. Legal Requirements

Software engineers must operate within local, national and international legal frameworks. Therefore, software engineers must know the legal requirements for the following:

- Registration and licensing, including examination, education, experience, and training requirements

- Contractual agreements
- Noncontractual legalities, such as those governing liability

Basic information on the international legal framework can be accessed from the World Trade Organization (WTO).

1.7.8. Trade Compliance

All software professionals must be aware of legal restrictions on the import, export, or re-export of goods, services, and technology in the jurisdictions in which they work. Such rules often concern export controls and classification; transfer of goods; acquisition of necessary governmental licenses for foreign use of hardware and software; services and technology by sanctioned nations, enterprises, or individual entities; and import restrictions and duties. Trade experts should be consulted for detailed compliance guidance.

1.7.9. Cybercrime

Cybercrime refers to any crime that involves a computer, computer software, computer networks, or embedded software controlling a system. The computer or software may have been used in the commission of a crime or have been the target of the crime. This category of crime includes fraud, unauthorized access, spam, obscene or offensive content, threats, harassment, theft of sensitive personal data or trade secrets, and use of one computer to damage or infiltrate other computers and automated system controls.

Computer and software users commit fraud by altering electronic data to facilitate illegal activity. Forms of unauthorized access include hacking, eavesdropping, and using computer systems in a way that is concealed from their owners. Many countries have laws that specifically cover cybercrimes, but many do not have effective statutes, making cybercrime difficult to prosecute in some cases. The software engineer has a professional obligation to consider the threat of cybercrime and to consider how the software

system's security will protect the software and user information from accidental or malicious access, use, modification, destruction, or disclosure.

Dark patterns are deceptive UI/UX interactions designed to mislead or trick users into making them do something they may not want to do. These patterns do not have the users' interests in mind and aim for exploitability rather than usability. Creating dark patterns is not good ethical practice. Software engineers should be responsible for their actions and be transparent with users instead of manipulating them.

1.7.10. Data Privacy

Software engineers should know that data privacy is a key legal requirement in many countries. The General Data Protection Regulation (GDPR), adopted on 14 April 2016, and enforceable since 25 May 2018, regulates data protection and privacy in the European Union (EU) and the European Economic Area (EEA). It also addresses the transfer of personal data outside the EU and EEA areas. The GDPR's primary aim is to enhance individuals' control and rights over their data and to simplify the regulatory environment for international business.

The regulation became a model for many national laws outside the EU, including the UK, Chile, Japan, Brazil, South Korea, Argentina, and Kenya. The California Consumer Privacy Act (CCPA), adopted on 28 June 2018, has many similarities with the GDPR.

1.8. Documentation

[1*, c10s5.8] [3*, c1s5] [4*] [5*, c32]

Providing clear, thorough, and accurate documentation is the responsibility of each software engineer. The adequacy of documentation is judged according to different criteria, based on stakeholder needs. Good documentation complies with accepted standards and guidelines. In particular, software engineers should document the following:

- Relevant facts
- Significant risks and trade-offs
- Warnings of undesirable or dangerous consequences from the use or misuse of the software
- Relevant information pertaining to attribute, license type, and sourcing

Software engineers should avoid:

- Certifying or approving unacceptable products
- Disclosing confidential information
- Falsifying facts or data

In addition, software engineers and their managers should provide the following documentation for other elements of the software development organization to use:

- Software requirements specifications, software design documents, details on the software engineering tools used, software test specifications and results, and details about the adopted software engineering methods
- Problems encountered during the development process

For external stakeholders (customers, users, others), software documentation should provide the following:

- Information needed to determine whether the software is likely to meet customer and user needs
- Description of safe and unsafe use of the software
- Explanation of how to protect sensitive information created by or stored using the software
- Clear identification of warnings and critical procedures

Software use may include installation, operation, administration, and performance of other functions by various groups of users and support personnel. If the customer will acquire ownership of the software source code or the right to modify the code, the software engineer

should provide documentation of the functional specifications, the software design, the test suite, and the necessary operating environment for the software. Documents should be kept for at least as long as the software product's life cycle or the time required by relevant organizational or regulatory requirements.

1.9. Trade-Off Analysis

[3*, c1s2, c10] [4*, c7s2, c13s4] [13*, c9s5.10]

A software engineer often has to choose between alternative problem solutions. The outcome of these choices is determined by the software engineer's professional evaluation of each alternative's risks, costs, and benefits in cooperation with stakeholders. The software engineer's evaluation is called *trade-off analysis*. Trade-off analysis notably identifies competing and complementary software requirements to prioritize the final requirements defining the software to be constructed. (See Requirements Negotiation in the Software Requirements KA and Determination and Negotiation of Requirements in the Software Engineering Management KA.)

When an ongoing software development project is late or over budget, a trade-off analysis is often conducted to decide which software requirements can be relaxed or dropped given the effects thereof. The first step in a trade-off analysis is establishing design goals (see Engineering Design in the Engineering Foundations KA) and setting the relative importance of those goals. This permits the identification of the solution that most nearly meets those goals; this means that the way the goals are stated is critically important.

Design goals may include minimizing monetary cost and maximizing reliability, performance, or other criteria on various dimensions. However, it is difficult to formulate a trade-off analysis of cost against risk, especially where primary production and secondary risk-based costs must be weighed against each other.

A software engineer must ethically conduct a trade-off analysis — notably by being

objective and impartial when selecting criteria for comparing alternative problem solutions and assigning weights or importance to these criteria. In addition, any conflict of interest must be disclosed upfront.

2. Group Dynamics and Psychology

Engineering work is often conducted in teams. A software engineer should interact cooperatively and constructively with others to first determine and then meet needs and expectations. Knowledge of group dynamics and psychology is an asset when interacting with customers, coworkers, suppliers, and subordinates to solve software engineering problems.

2.1. Dynamics of Working in Teams/Groups

[3*, c1s6] [14*, c1s3.5, c10]

Software engineers must work with others. On the one hand, they work internally in engineering teams; on the other hand, they work with customers, members of the public, regulators, and other stakeholders. Performing teams — those who demonstrate a consistent quality of work and progress toward goals — are cohesive and possess a cooperative, honest and focused atmosphere. Individual and team goals are aligned so the members naturally commit to and feel ownership of shared outcomes.

Team members facilitate this atmosphere by being intellectually honest, using group thinking, admitting ignorance, and acknowledging mistakes. They share responsibility, rewards, and workload fairly. They communicate clearly and directly to one another and in documents and source code so information is accessible to everyone. Peer reviews about work products are framed in a constructive and nonpersonal way. (See Reviews and Audits in the Software Quality KA.) This allows all the members to pursue a continuous improvement and growth cycle without personal risk. Members of cohesive teams demonstrate respect for one another and their leader.

One point to emphasize is that software

engineers must be able to work in multidisciplinary environments and varied application domains. Because software is everywhere, from phones to cars, it affects people's lives far beyond the more traditional concept of software made for information management in a business environment.

2.2. Individual Cognition

[3*, c1s6.5] [5*, c33]

Engineers want to solve problems. Every engineer strives to solve problems effectively and efficiently. However, the limits and processes of individual cognition affect problem-solving. Individual cognition plays a prominent role in problem-solving in software engineering, in part because of the highly abstract nature of software itself.

An individual's (in particular, a software engineer's) ability to decompose a problem and creatively develop a solution can be inhibited by the following:

- The need for more knowledge
- Subconscious assumptions
- The volume of data
- Fear of failure or the consequence of failure
- Culture, either of the application domain or the organization
- Lack of ability to express the problem
- Perceived working atmosphere
- The individual's emotional status

The effects of these inhibiting factors can be reduced by cultivating good problem-solving habits that minimize the impact of misleading assumptions. The ability to focus is crucial, as is intellectual humility. Both allow a software engineer to suspend personal considerations and consult with others freely, which is especially important when working in teams.

Engineers use basic methods to facilitate problem-solving. (See Problem-Solving Techniques in the Computing Foundations KA.) Breaking down problems and solving them one piece at a time reduces cognitive overload. By taking advantage of professional

curiosity and pursuing continuous professional development, engineers gain skills and knowledge. Reading, networking, and experimenting with new tools, techniques and methods are all valid means of professional development.

2.3. Dealing with Problem Complexity

[3*, c3s2] [4*, c1s1, c20s1-s5] [5*, c33]

Many, if not most, software engineering problems are too complex and difficult to address as a whole or to be tackled by individual software engineers. When such circumstances arise, engineers typically use teamwork and problem decomposition. (See Problem-Solving Techniques in the Computing Foundations KA.)

Teams work together to deal with large, complex problems by sharing burdens and drawing on one another's knowledge and creativity. When software engineers work in teams, individual engineers' different views and abilities complement one another and help build a solution otherwise difficult to come by. Some teamwork examples in software engineering are pair programming (see Agile Methods in the Software Engineering Models and Methods KA) and code review (see Reviews and Audits in the Software Quality KA).

2.4. Interacting with Stakeholders [4*]

The success of a software engineering endeavor depends on positive interactions with stakeholders. Stakeholders should provide support, information, and feedback at all stages of the software life cycle. For example, during the early stages, it is critical to identify all stakeholders and discover how the product will affect them to properly capture a sufficient definition of stakeholder requirements.

In Agile software development, the involvement of stakeholders is even more essential than in other types of development. First, during development, stakeholders may provide feedback on specifications or early versions of the software, changes of priority, and clarification of detailed or new software

requirements. Last, during software maintenance and until the end of product life, stakeholders can provide feedback on evolving or new requirements and problem reports so the software can be extended and improved. Clearly, regular stakeholder involvement will lead to a better application. It is vital to maintain open and productive communication with stakeholders during the software product's life cycle.

2.5. *Dealing with Uncertainty and Ambiguity* [4*, c4s1, c4s4, c11s5, c24s5] [14*, c9s4]

As with engineers in other fields, software engineers must often deal with and resolve uncertainty and ambiguities while providing services and developing products. The software engineer must reduce or eliminate any lack of clarity that is an obstacle to performing work.

Often, uncertainty reflects a lack of knowledge. If that is the case, investigating the issue by reading formal sources such as textbooks and professional journals, interviewing stakeholders, or consulting with teammates and peers can likely solve the problem.

When uncertainty or ambiguity cannot be overcome easily, software engineers or organizations may regard it as a project risk. When this is the case, work estimates or pricing are adjusted to mitigate the anticipated cost of addressing it. (See Risk Management in the Software Engineering Management KA.)

2.6. *Dealing with Equity, Diversity, and Inclusivity* [4*] [14*, c10s7]

The equity, diversity, and inclusivity environment can affect a group's dynamics. This is especially true when the group is geographically separated or communication is infrequent because such separation elevates the importance of each contact. Intercultural communication is even more difficult if the difference in time zones makes oral communication less frequent.

Multicultural environments are prevalent in software engineering, perhaps more

than in other engineering fields, because of the strong trend of international outsourcing and the easy shipment of software components instantaneously around the globe. For example, it is common for a software project to be divided into pieces across national and cultural borders. It is also common for a software project team to consist of people from diverse cultural backgrounds.

For a software project to succeed, team members must embrace tolerance of different cultural and social norms, acknowledging that not all societies have the same social expectations. The support of leadership and management can facilitate tolerance and understanding. More frequent communication, including face-to-face meetings, can help mitigate geographical and cultural divisions, promote cohesiveness, and raise productivity. Also, communicating with teammates in their native language could be beneficial.

In the software industry, gender bias is still prevalent. Implementing broader recruiting strategies, specific and measurable performance evaluation criteria, and transparent procedures for assigning compensation can reduce gender inequality in the software industry. These trends can contribute to building a diverse environment for all software engineers, regardless of their gender.

3. Communication Skills

A software engineer must communicate well, both orally and in reading and writing. To meet software requirements and deadlines, engineers must establish clear communication with customers, supervisors, coworkers, and suppliers. Optimal problem-solving is made possible through the ability to investigate, comprehend and summarize information. Customer product acceptance and safe product use depend on relevant training and documentation. The software engineer's career success is affected by consistently providing oral and written communication effectively and on time.

3.1. *Reading, Understanding, and Summarizing* [4*, c4s5] [5*, c33s3]

Software engineers must be able to read and understand technical material. Technical material includes reference books, manuals, research papers, online sources and program source code.

Reading is not only a primary way of improving skills but also a way of gathering information for completing engineering goals. A software engineer sifts through accumulated information, focusing on the pieces that will be most helpful. Customers may request that a software engineer summarize the results of such information-gathering for them, simplifying or explaining it so that they can make the final choice among competing solutions.

Reading and comprehending source code are also components of information-gathering and problem-solving. For example, when engineers modify, extend or rewrite software, they must understand both its implementation, directly derived from the presented code, and its design, which must often be inferred.

3.2. *Writing* [3*, c1s5] [4*, c4s2-s3]

Software engineers can produce written products requested by customers or required by generally accepted practice. These written products may include source code, software project plans, software requirement documents, risk analyses, software design documents, software test plans, user manuals, technical reports and evaluations, justifications, diagrams and charts, and so forth.

Clear, concise writing is important because writing is often the primary method of communication among relevant parties. In all cases, written software engineering products must be accessible, understandable, and relevant to their intended audience.

3.3. *Team and Group Communication* [3*, c1s6.8] [4*, c22s3] [5*, c27s1] [14*, c10s4]

Effective communication among team and group members is essential to a collaborative

software engineering effort. Stakeholders must be consulted; decisions must be made, and plans must be generated. The greater the number of team and group members, the greater the need to communicate.

However, the number of communication paths grows quadratically with the addition of each team member. Furthermore, team members are unlikely to communicate with anyone perceived to be removed from them by more than two degrees (levels). This problem can be more serious when software engineering endeavors or organizations are spread across national and continental borders.

Some communication can be accomplished in writing. Software documentation is a common substitute for direct interaction. Email is another, but although it is useful, it is not always enough. Also, if one receives too many messages, it becomes difficult to identify the important information. Increasingly, organizations are using enterprise collaboration tools to share information. In addition, electronic information stores, accessible to all team members for organizational policies, standards, common engineering procedures, and project-specific information, can be beneficial.

Some software engineering teams focus on face-to-face interaction and promote such interaction through office space arrangements. Although private offices improve individual productivity, other arrangements, such as co-locating team members in physical or virtual spaces and providing communal work areas, can boost collaborative efforts.

3.4. *Presentation Skills* [3*, c1s5] [4*, c22] [14*, c10s7-c10s8]

Software engineers rely on their presentation skills during software life cycle processes. For example, software engineers may walk customers and teammates through software requirements during the phase and conduct formal requirements reviews. (See Requirement Reviews in the Software Requirements KA.) During and after software design, software construction, and software maintenance, software engineers lead

reviews, product walkthroughs (see Review and Audits in the Software Quality KA), and training. These require the ability to present technical information to groups and solicit ideas or feedback.

Therefore, the software engineer's ability to convey concepts effectively in a presentation

influences product acceptance, management, and customer support. It also influences the ability of stakeholders to comprehend and assist in the product effort. This knowledge needs to be archived in slides, knowledge write-ups, technical white papers, and other material used for knowledge creation.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Bott et al. 2000 [1*]	Voland, 2003 [3*]	Sommerville, 2016 [4*]	McConnel, 2004 [5*]	Tockey, 2004 [13*]	Fairley, 2009 [14*]
1. Professionalism						
<i>1.1. Accreditation, Certification and Qualification, and Licensing</i>	c1s5, c1s10		c12s10			
<i>1.2. Codes of Ethics and Professional Conduct</i>	c1s7-s9, c10s2, App		c1s2			
<i>1.3. Nature and Role of Professional Societies</i>	c2s3		c1s2	c35s1		
<i>1.4. Nature and Role of Software Engineering Standards</i>	c10s2,		*	c32s6		
<i>1.5. Economic Impact of Software</i>		c1s1, c10s8	c1s1		*	
<i>1.6. Employment Contracts</i>	c6, c7		*			
<i>1.7. Legal Issues</i>	c6, c11	c5s3-s4,	c12s3, c13s2			
<i>1.8. Documentation</i>	c10s5	c1s5	*	c32		
<i>1.9. Trade-Off Analysis</i>		c1s2, c10	c7s2, c13s4		c9s5-10	
2. Group Dynamics and Psychology						
<i>2.1. Dynamics of Working in Teams/Groups</i>		c1s6				c1s3-5, c10
<i>2.2. Individual Cognition</i>		c1s6		c33		
<i>2.3. Dealing with Problem Complexity</i>		c3s2	c1s1, c20s1-s5			
<i>2.4. Interacting with Stakeholders</i>			*			
<i>2.5. Dealing with Uncertainty and Ambiguity</i>			c4s1,c4s4, c11s5c24s5			c9s4

2.6. <i>Dealing with Equity, Diversity, and Inclusivity</i>			*			c10s7
3. Communication Skills						
3.1. <i>Reading, Understanding, and Summarizing</i>			c4s5	c33s3		
3.2. <i>Writing</i>		c1s5	c4s2-s3			
3.3. <i>Team and Group Communication</i>		c1s6	c22s3	c27s1		c10s4
3.4. <i>Presentation Skills</i>		c1s5	c22			c10s7-s8

FURTHER READINGS

G.M. Weinberg, *The Psychology of Computer Programming* [15].

This was the first major book to address programming as an individual and team effort; it has become a classic in the field.

Kinney and Lange, P.A., *Intellectual Property Law for Business Lawyers* [16].

This book covers IP laws in the US. It not only talks about what the IP law is; it also explains why it looks the way it does.

REFERENCES

[1*] F. Bott et al., *Professional Issues in Software Engineering*, 3rd ed., Taylor & Francis, 2000.

[2] Appendix B of this Guide.

[3*] G. Voland, *Engineering by Design*, 2nd ed., Prentice-Hall, 2003.

[4*] I. Sommerville, *Software Engineering*, 10th ed., Addison-Wesley, 2016.

[5*] S. McConnell, *Code Complete*, 2nd ed., Microsoft Press, 2004.

[6] 25 Years Washington Accord, IEC, 2014.

[7] EUR-ACE, 2017.

[8] ISO/IEC 24773-1, Software and Systems Engineering – Certification of Software and System Engineering Professionals — Part 1: General Requirements.

[9] ISO/IEC 24773-4 Software and Systems Engineering — Certification of Software and Systems Engineering Professionals — Part 4: Software engineering.

[10] ACM Code of Ethics and Professional Conduct, 2018.

[11] IEEE Code of Ethics, 2020.

[12] IFIP Code of Ethics and Professional Conduct, 2021.

[13*] S. Tockey, *Return on Software: Maximizing the Return on Your Software Investment*, Addison-Wesley, 2004.

[14*] R. E. Fairley, *Managing and Leading Software Projects*, Wiley-IEEE Computer Society Press, 2009.

[15] G. M. Weinberg, *The Psychology of Computer Programming: Silver Anniversary Edition*, Dorset House, 1998.

[16] Kinney and Lange, P.A., *Intellectual Property Law for Business Lawyers*, Thomson West, 2013.